Day 1 Study Notes

Python Refresher & Data Basics

AI Training Program • 5th October 2026

Topic	What it covers
1. NumPy & Pandas	Arrays, statistics, DataFrame creation, selection, filtering, new columns, sorting, grouping
2. Visualization	Matplotlib bar/line charts, Seaborn box plots, choosing the right chart
3. Types of Data	Structured (CSV, SQL), semi-structured (JSON), unstructured (text)
4. Data Collection & Storage	Application databases (SQLite), user input files, organizational data stores
5. Revision	One-page cheat sheet + 50 MCQs with answer key

Each topic links back to the scripts you wrote in your Day 1 folders (Database_demo, pandas and matplotlib exercise, python basics, python_project1) so you can open the file and recollect what you did.

Topic 1 — NumPy & Pandas: DataFrame Manipulation

1.1 NumPy — Numerical Python

NumPy is the core Python library for working with numbers. Its main object is the **ndarray** (N-dimensional array): a grid of values that all have the same data type. Because arrays are stored compactly and operations run in optimized C code, NumPy is much faster than plain Python lists for maths.

- **Homogeneous:** every element has the same type (e.g., all floats).
- **Vectorized:** an operation applies to every element at once, no loop needed (`prices * 2`).
- **Foundation:** pandas, Matplotlib and most ML libraries are built on NumPy arrays.

Creating arrays and basic statistics

```
import numpy as np

prices = np.array([120, 250, 90, 300, 180])

print(prices.mean())   # average      -> 188.0
print(prices.max())    # largest      -> 300
print(prices.min())    # smallest     -> 90
print(prices.sum())    # total        -> 940
print(prices.std())    # standard deviation
print(np.median(prices))  # middle value -> 180.0
```

Useful array operations

```
prices[0]          # first element (indexing starts at 0)
prices[1:3]        # slicing -> elements at index 1 and 2
prices * 1.1       # add 10% to every price (vectorized)
prices[prices > 150]   # boolean filtering -> [250, 300, 180]
prices.shape       # (5,)  -> one dimension with 5 values
np.arange(0, 10, 2)    # [0 2 4 6 8]
np.zeros(3), np.ones(3)
```

List (Python)	NumPy array
Can mix types (int, str, ...)	One data type for all elements
[1,2]*2 → [1,2,1,2] (repeats)	np.array([1,2])*2 → [2,4] (maths)
Slower for large numeric data	Fast, memory-efficient
No built-in mean/std	Built-in .mean(), .std(), .max() ...

✎ From your practice files

• **python basics/numpy_basics.py** and **python_project1/numpy_basics.py** — you created a NumPy array of prices and printed its **mean, maximum and minimum**.
• **pandas and matplotlib exercise/pandas_exercise.py** — you also calculated statistics on the employee scores using NumPy functions.

1.2 Pandas — Working with Tables

Pandas is the main library for tabular (row-and-column) data. It has two key structures:

Structure	Meaning	Example
Series	One labeled column of values (1-D)	df['Department']
DataFrame	A 2-D table: rows + labeled columns (like an Excel sheet)	the full employee table

Creating a DataFrame from a dictionary

```
import pandas as pd

data = {
    "Employee": ["Asha", "Ravi", "Meena", "John"],
    "Department": ["HR", "IT", "Finance", "IT"],
    "Training_Hours": [12, 20, 15, 8],
    "Assessment_Score": [78, 92, 65, 81],
    "Completed": [True, True, False, True],
}
df = pd.DataFrame(data)   # keys become column names
```

Inspecting data — always the first step

Command	What it shows
df.head() / df.head(4)	First 5 rows (default) / first 4 rows
df.tail()	Last 5 rows
df.shape	Tuple (rows, columns), e.g. (8, 5)
df.columns	List of column names
df.dtypes	Data type of each column (int64, float64, object, bool)
df.info()	Columns, non-null counts, types, memory
df.describe()	Count, mean, std, min, quartiles, max for numeric columns

Selecting columns and rows

```
df["Employee"]                          # one column -> Series
df[["Employee", "Assessment_Score"]]    # several columns -> DataFrame (double brackets!)

df.loc[0]               # row with LABEL 0
df.iloc[0]              # row at POSITION 0
df.loc[0:2, "Employee"]  # loc slices INCLUDE the end label (0,1,2)
df.iloc[0:2, 0]          # iloc slices EXCLUDE the end (0,1)
```

Filtering rows (boolean conditions)

```
df[df["Department"] == "IT"]                # only IT employees
df[df["Assessment_Score"] > 80]             # high scorers
df[df["Completed"] == True]                 # completed training

# Combine conditions: use & (and), | (or), ~ (not) and WRAP EACH in ( )
df[(df["Department"] == "IT") & (df["Assessment_Score"] > 80)]
df[(df["Department"] == "HR") | (df["Department"] == "Finance")]
df[df["Department"].isin(["HR", "Finance"])]   # shorter version of the line above
```

Adding calculated columns

```
import numpy as np
df["Passed"] = np.where(df["Assessment_Score"] >= 75, "Pass", "Fail")

# another way using apply + a function
df["Fine_Status"] = df["Fine"].apply(lambda f: "Has fine" if f > 0 else "No fine")

df["Total"] = df["Quantity"] * df["Price"]    # arithmetic between columns
```

Aggregating, sorting and grouping

```
df["Assessment_Score"].mean()     # average
df["Assessment_Score"].min()      # minimum
df["Assessment_Score"].max()      # maximum
df["Training_Hours"].sum()        # total

df.sort_values("Assessment_Score")                    # ascending (low -> high)
df.sort_values("Assessment_Score", ascending=False)   # descending (high -> low)

df.groupby("Department")["Assessment_Score"].mean()   # average score per department
df.groupby("Department")["Training_Hours"].sum()      # total hours per department
df.groupby("Department").agg({"Assessment_Score": ["mean", "max"]})

df["Department"].value_counts()   # how many employees in each department
df["Completed"].value_counts()    # how many completed vs not
```

How groupby works — split → apply → combine: pandas splits rows into groups by the key (HR, IT, Finance), applies a function (mean, sum, count) to each group, then combines the results into a new table.

Reading data from files and databases into pandas

Source	Function
CSV file	pd.read_csv('library_loans.csv')
Excel file	pd.read_excel('file.xlsx')
JSON (flat)	pd.read_json('file.json')
JSON (nested)	pd.json_normalize(records)
SQL database	pd.read_sql_query('SELECT * FROM bookings', conn)
Save to CSV	df.to_csv('out.csv', index=False)

✎ From your practice files

• **python basics/data_basics.py** — product sales DataFrame: selected and sorted data, calculated totals, grouped sales by product.
• **pandas and matplotlib exercise/pandas_exercise.py** — 8 employees (HR, IT, Finance): head(4), selected columns, filtered by department/score/completion, combined conditions, added **Passed** (score ≥ 75), averages, min/max, sorting, groupby department, value_counts for department and completion.
• **python_project1/structured_assignment.py** — library_loans.csv: selected loan fields, filtered fines, added a fine-status column, average borrowed days, total fines by book category.
• **python_project1/structred_demo.py** — ev_sessions.csv: inspected data, columns, shape, types.

⚠ Remember / common mistakes

• Multiple columns need **double brackets**: df[['a','b']], not df['a','b'].
• Use **&** and **|** in pandas filters, not Python's *and*/*or*; put every condition in parentheses.
• **loc** = labels (end included); **iloc** = positions (end excluded).
• sort_values() returns a **new** DataFrame — assign it (df = df.sort_values(...)) if you want to keep the order.
• == compares; = assigns. df[df['Dept'] = 'IT'] is an error.

Topic 2 — Visualization (Matplotlib & Seaborn)

Visualization turns numbers into pictures so patterns, comparisons and outliers are easy to see. Matplotlib is the base plotting library; Seaborn sits on top of it and makes good-looking statistical charts with less code.

2.1 Matplotlib

Import convention: `import matplotlib.pyplot as plt`. Every chart follows the same pattern: **create data → draw plot → add title/labels → plt.show()**.

```
import matplotlib.pyplot as plt

avg_scores = df.groupby("Department")["Assessment_Score"].mean()

plt.figure(figsize=(6, 4))                # chart size in inches
plt.bar(avg_scores.index, avg_scores.values, color="steelblue")
plt.title("Average Assessment Score by Department")
plt.xlabel("Department")
plt.ylabel("Average Score")
plt.tight_layout()
plt.show()                                # opens the chart window

# Shortcut: pandas can plot directly using Matplotlib
df.groupby("Department")["Training_Hours"].sum().plot(kind="bar")
plt.show()
```

Function	Chart	Use it for
plt.bar(x, y)	Bar chart	Comparing values across categories
plt.barh(x, y)	Horizontal bar	Long category names
plt.plot(x, y)	Line chart	Trends over time
plt.scatter(x, y)	Scatter plot	Relationship between two numeric variables
plt.hist(data)	Histogram	Distribution of one numeric variable
plt.pie(values)	Pie chart	Parts of a whole (few categories)
plt.title / xlabel / ylabel	—	Chart title and axis labels
plt.legend()	—	Show labels for multiple series
plt.savefig('chart.png')	—	Save chart to a file (call before show)

2.2 Seaborn

Import convention: `import seaborn as sns`. Seaborn works directly with DataFrames — you pass `data=df` and column names for x and y.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(data=df, x="Department", y="Assessment_Score")
plt.title("Assessment Scores by Department")
plt.show()

sns.barplot(data=df, x="Department", y="Assessment_Score")   # bar = mean per group
sns.countplot(data=df, x="Department")                       # counts rows per category
sns.histplot(data=df, x="Assessment_Score")                  # distribution
sns.scatterplot(data=df, x="Training_Hours", y="Assessment_Score")
```

Reading a box plot

- **Line inside the box** = median (middle value).
- **Box edges** = Q1 (25th percentile) and Q3 (75th percentile); the box height is the **IQR** (interquartile range).
- **Whiskers** = spread of the rest of the data (typically up to 1.5 × IQR).
- **Dots beyond whiskers** = outliers.
- Comparing boxes side-by-side shows which department scores higher and which is more spread out.

Matplotlib	Seaborn
Low-level, full control over every element	High-level, built on Matplotlib
More code for styled charts	Attractive defaults with one line
Works with lists/arrays	Designed for DataFrames (data=, x=, y=)
bar, line, scatter, hist, pie	boxplot, barplot, countplot, histplot, heatmap

✎ From your practice files

• **pandas_exercise.py** opened three charts: a Matplotlib bar chart of **average score per department**, a Matplotlib bar chart of **total training hours per department**, and a Seaborn **box plot** comparing department scores.
• **data_basics.py** / **python basics.py** — bar chart of product sales.
• Charts open in a window; the script continues only after you close it.

⚠ Remember / common mistakes

• Always add a title and axis labels — a chart without labels is hard to read.
• Call plt.savefig() **before** plt.show(), otherwise the saved image may be blank.
• Pie charts are poor with many categories; use a bar chart instead.

Topic 3 — Types of Data

Data is grouped by how organized it is. The type decides which tools you use to read and analyse it.

	Structured	Semi-structured	Unstructured
Organization	Fixed rows & columns, predefined schema	Tags / keys, flexible and often nested	No predefined model
Examples	CSV, Excel, SQL tables	JSON, XML, YAML, emails, logs	Text docs, PDFs, images, audio, video
Python tools	pandas read_csv, sqlite3, read_sql_query	json module, pd.json_normalize	open().read(), split(), string methods, NLP
Ease of analysis	Easiest	Medium	Hardest
Your files	ev_sessions.csv, library_loans.csv, cinema.db	support_tickets.json, festival_events.json	incident_notes.txt, wildlife_report.txt

3.1 Structured data

Structured data has a fixed **schema**: every record has the same columns and each column has a defined type. It fits naturally into tables, spreadsheets and relational databases, and can be queried with SQL or pandas.

```
import pandas as pd
loans = pd.read_csv("library_loans.csv")
print(loans.shape, loans.dtypes)
print(loans.groupby("Category")["Fine"].sum())   # total fines per book category
```

3.2 Semi-structured data

Semi-structured data has some organization through **keys or tags** but no rigid table schema. Records can have different fields (optional keys) and values can be **nested** (an object inside an object, or a list). JSON is the most common example.

```
import json
import pandas as pd

with open("support_tickets.json") as f:
    tickets = json.load(f)              # JSON -> Python list of dicts

first = tickets[0]
print(first["customer"]["name"])        # access NESTED data
print(first.get("priority", "Not set")) # optional key: .get() avoids KeyError

df = pd.json_normalize(tickets)         # flatten nested JSON into a table
print(df.columns)   # nested keys become columns like 'customer.name', 'customer.city'
```

JSON	Python
object { }	dict
array [ ]	list

JSON	Python
string	str
number	int / float
true / false	True / False
null	None

3.3 Unstructured data

Unstructured data has no fixed format — free text, images, audio, video. It makes up the majority of the world's data. To analyse it you first have to extract structure (word counts, keywords, labels).

```
with open("wildlife_report.txt") as f:
    text = f.read()

words = text.split()
print("Total words:", len(words))
print("Count of 'tiger':", text.lower().count("tiger"))
print("Appears?", "tiger" in text.lower())

for line in text.splitlines():          # print matching lines
    if "tiger" in line.lower():
        print(line)
```

✎ From your practice files

• **Structured:** structred_demo.py (ev_sessions.csv), structured_assignment.py (library_loans.csv), database_demo.py (cinema.db).
• **Semi-structured:** semistructred.py (support_tickets.json — nested customer details, optional priority, flattening) and semi_structured_assignment.py (festival_events.json — nested location, optional features, normalize).
• **Unstructured:** unstructred_demo.py (incident_notes.txt — word count, search a term) and unstructured_assignment.py (wildlife_report.txt — count words & occurrences, check presence, print matching lines).

⚠ Remember / common mistakes

• json.load(f) reads from a **file**; json.loads(s) reads from a **string**.
• Use dict.get('key', default) for keys that may be missing.
• Lower-case the text before counting so 'Tiger' and 'tiger' match.
• Spreadsheets with free-text comments mix types: the columns are structured, the comment text is unstructured.

Topic 4 — Data Collection & Storage

Data collection is gathering data from its sources; **data storage** is keeping it safely so it can be retrieved and analysed later. In AI projects, the quality of collected data directly decides model quality.

Source	Description	Examples
Application databases	Databases behind apps that record every transaction	Bookings, orders, loans, user accounts (SQLite, MySQL, PostgreSQL, MongoDB)
User input files	Files supplied or uploaded by users	CSV/Excel uploads, JSON exports, text reports, forms, input()
Organizational data stores	Central company repositories for reporting and analytics	Data warehouse, data lake, data mart, shared drives, cloud storage
Other sources	External or automatic collection	APIs, web scraping, sensors/IoT, surveys, logs

4.1 Application databases

A **relational database** stores data in tables linked by keys and is queried with **SQL** (Structured Query Language). **SQLite** is a lightweight relational database stored in a single file — Python includes it through the built-in `sqlite3` module, so nothing has to be installed.

Relational (SQL)	NoSQL
Tables with fixed schema	Documents, key-value, graphs; flexible schema
SQLite, MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra
Best for structured, consistent data	Good for semi-structured / fast-changing data

The sqlite3 workflow

Step	Code	Purpose
1. Connect	conn = sqlite3.connect('cinema.db')	Opens the file; creates it if it doesn't exist
2. Cursor	cur = conn.cursor()	Object that sends SQL to the database
3. Execute	cur.execute(sql)	Run CREATE / INSERT / SELECT
4. Commit	conn.commit()	Save changes permanently (needed after INSERT/UPDATE/DELETE)
5. Fetch	cur.fetchall()	Get query results as a list of tuples
6. Close	conn.close()	Release the database

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("cinema.db")
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS bookings (
    booking_id INTEGER PRIMARY KEY,
    movie      TEXT,
    seats      INTEGER,
    amount     REAL
)
""")

cur.execute("INSERT INTO bookings (movie, seats, amount) VALUES (?, ?, ?)",
            ("Solar Drift", 2, 400.0))
cur.execute("INSERT INTO bookings (movie, seats, amount) VALUES (?, ?, ?)",
            ("Midnight Code", 3, 600.0))
conn.commit()                                   # save the inserts

cur.execute("SELECT * FROM bookings")
print(cur.fetchall())   # [(1, 'Solar Drift', 2, 400.0), (2, 'Midnight Code', 3, 600.0)]

df = pd.read_sql_query("SELECT * FROM bookings", conn)   # same data as a DataFrame
print(df)
conn.close()
```

Key SQL terms

Term	Meaning
CREATE TABLE IF NOT EXISTS	Create the table only once; no error if it already exists
INTEGER / TEXT / REAL	Whole numbers / strings / decimal numbers
PRIMARY KEY	Column that uniquely identifies each row; SQLite auto-assigns INTEGER PRIMARY KEY values
INSERT INTO ... VALUES	Add a new row
SELECT * FROM t	Read all columns from a table
WHERE	Filter rows: SELECT * FROM bookings WHERE seats > 2
ORDER BY	Sort results: ORDER BY amount DESC
UPDATE / DELETE	Change / remove existing rows
? placeholders	Parameterized query — safe way to insert values (prevents SQL injection)

4.2 User input files

Many datasets arrive as files: a manager uploads an Excel sheet, a system exports a CSV, a partner sends JSON. Python reads them with `open()` (best inside a `with` block, which closes the file automatically) or directly with pandas. Data typed by a user at runtime comes from `input()`, which always returns a string.

```
with open("incident_notes.txt", "r") as f:   # 'r' read, 'w' write (overwrite), 'a' append
    content = f.read()

age = int(input("Enter age: "))   # input() returns str -> convert for numbers

df = pd.read_csv("ev_sessions.csv")          # user-supplied CSV
```

- **Validate** user files: check columns exist, types are correct, and no required values are missing.
- **Relative paths** ("library_loans.csv") are resolved from the current working folder — this is why you run scripts from the project folder.

4.3 Organizational data stores

Store	What it holds	Typical use
Data warehouse	Cleaned, structured, historical data from many systems, organised for querying	Business reports, dashboards (OLAP)
Data lake	Raw data in any format — structured, semi-structured, unstructured	Data science, ML, storing everything cheaply
Data mart	A smaller subset of the warehouse for one department	Sales or HR team analytics
Operational DB	Live day-to-day transactions	Running the app (OLTP)
File shares / cloud storage	Documents, spreadsheets, media	Shared drives, S3, Google Drive

OLTP vs OLAP: OLTP (Online Transaction Processing) systems handle many small, fast reads/writes — the application database. OLAP (Online Analytical Processing) systems handle large analytical queries — the data warehouse. **ETL** (Extract, Transform, Load) is the process that moves data from operational systems into the warehouse.

✎ From your practice files

• **Database_demo/database_demo.py** — connected to cinema.db, created the **bookings** table (booking_id, movie, seats, amount), inserted *Solar Drift* and *Midnight Code*, committed, printed rows with fetchall(), then printed them as a pandas DataFrame.
• **python_project1/database_demo.py** — read the existing bookings table into pandas (the create/insert code was commented out).
• Your CSV, JSON and TXT exercises are examples of **user input files**.

⚠ Remember / common mistakes

• Forgetting **conn.commit()** means inserts are lost when the program ends.
• Re-running a script that always INSERTs creates **duplicate rows** — delete cinema.db to start fresh.
• Use ? placeholders instead of building SQL with f-strings.

Quick Revision Cheat Sheet

Task	Code
Import libraries	import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
Array stats	arr.mean(), arr.max(), arr.min(), arr.sum(), arr.std()
Make DataFrame	pd.DataFrame(dict)
Inspect	df.head(), df.shape, df.columns, df.dtypes, df.info(), df.describe()
Select	df['c'], df[['a','b']], df.loc[label], df.iloc[pos]
Filter	df[(df['a'] > 5) & (df['b'] == 'X')]
New column	df['P'] = np.where(cond, 'Pass', 'Fail')
Sort	df.sort_values('c', ascending=False)
Group	df.groupby('Dept')['Score'].mean()
Count	df['Dept'].value_counts()
Bar chart	plt.bar(x, y); plt.title(); plt.show()
Box plot	sns.boxplot(data=df, x='Dept', y='Score')
Read CSV	pd.read_csv('file.csv')
Read JSON	json.load(open('f.json')); pd.json_normalize(data)
Read text	open('f.txt').read(); len(text.split()); text.count('w')
SQLite	connect → cursor → execute → commit → fetchall → close
SQL to pandas	pd.read_sql_query('SELECT * FROM t', conn)

Practice MCQs (1 – 50)

Try every question first, then check the answer key on the next pages.

1. Which library is mainly used for fast numerical arrays in Python?

A) sqlite3
B) NumPy
C) pandas
D) seaborn

2. What does np.array([10, 20, 30]).mean() return?

A) 30
B) 20.0
C) 10
D) 60

3. What is the result of np.array([1, 2]) * 2?

A) [2, 4]
B) [1, 2, 1, 2]
C) Error
D) [3, 4]

4. All elements of a NumPy array must be of:

A) Integer type only
B) String type only
C) Different types
D) The same data type

5. What is the standard alias used when importing pandas?

A) pd
B) pn
C) pa
D) np

6. A two-dimensional labeled table in pandas is called a:

A) ndarray
B) Tuple
C) Series
D) DataFrame

7. By default, df.head() shows how many rows?

A) 5
B) All rows
C) 10
D) 3

8. df.shape returns:

A) Data types
B) Number of columns only
C) A tuple (rows, columns)
D) Column names

9. Which correctly selects the columns 'Employee' and 'Department'?

A) df[['Employee', 'Department']]
B) df.select('Employee', 'Department')
C) df['Employee', 'Department']
D) df('Employee', 'Department')

10. Which code keeps only employees with Assessment_Score above 80?

A) df['Assessment_Score' > 80]
B) df.where('> 80')
C) df.filter(> 80)
D) df[df['Assessment_Score'] > 80]

11. Which operator combines two pandas filter conditions with AND?

A) +
B) and
C) &&
D) & (with each condition in parentheses)

12. df.iloc[0] returns:

A) An error
B) The first row by position
C) The first column
D) The row whose label is 0

13. Which method sorts a DataFrame by a column's values?

A) df.arrange()
B) df.sort_values()
C) df.sort()
D) df.order_by()

14. To sort from highest to lowest you pass:

A) ascending=False
B) order='down'
C) reverse=True
D) desc=True

15. df.groupby('Department')['Assessment_Score'].mean() returns:

A) Average score for each department
B) Highest score per department
C) Number of employees per department
D) Overall average score

16. Which counts how many employees are in each department?

A) df['Department'].count()
B) df['Department'].value_counts()
C) df.count('Department')
D) len(df['Department'])

17. Which creates a 'Passed' column with 'Pass' for scores of 75 or more?

A) df['Passed'] = np.where(df['Assessment_Score'] >= 75, 'Pass', 'Fail')
B) df['Passed'] == 'Pass'
C) df['Passed'] = if df['Assessment_Score'] >= 75
D) df.add('Passed', 75)

18. df.describe() shows:

A) Memory usage only
B) Column names only
C) Summary statistics of numeric columns
D) First 5 rows

— Visualization —

19. Which is the standard import for Matplotlib plotting?

A) from matplotlib import sns
B) import matplotlib as plt
C) import pyplot as mp
D) import matplotlib.pyplot as plt

20. Which function displays the chart window?

A) plt.draw_chart()
B) plt.open()
C) plt.show()
D) plt.display()

21. Seaborn is built on top of:

A) NumPy only
B) SQLite
C) Matplotlib
D) JSON

22. Best chart for comparing total training hours across departments:

A) Bar chart
B) Pie chart with 20 slices
C) Line chart
D) Scatter plot

23. In a box plot, the line inside the box shows the:

A) Mode
B) Median
C) Maximum
D) Mean

24. The box in a box plot stretches from:

A) Mean to median
B) Minimum to maximum
C) 0 to 100
D) Q1 to Q3 (the IQR)

25. Which chart is best for showing a trend over time?

A) Pie chart
B) Line chart
C) Box plot
D) Count plot

26. plt.xlabel('Department') sets:

A) The legend
B) The x-axis label
C) The y-axis label
D) The chart title

27. Which Seaborn function draws a box plot?

A) sns.box()
B) sns.boxplot()
C) sns.whisker()
D) sns.plotbox()

28. Points drawn outside the whiskers of a box plot represent:

A) Median values
B) Missing values
C) Outliers
D) Averages

— Types of Data —

29. Data stored in rows and columns with a fixed schema is:

A) Unstructured
B) Semi-structured
C) Raw media
D) Structured

30. A JSON file such as support_tickets.json is an example of:

A) Semi-structured data
B) Structured data
C) Unstructured data
D) Binary data

31. Free-text incident notes in a .txt file are:

A) Relational
B) Semi-structured
C) Unstructured
D) Structured

32. Which of these is NOT structured data?

A) An SQL table
B) An Excel sheet of sales
C) A CSV file
D) A video recording

33. Which pandas function flattens nested JSON into a table?

A) pd.read_nested()
B) pd.json_normalize()
C) pd.read_text()
D) pd.flatten()

34. json.load(f) is used to:

A) Read JSON from a file into Python objects
B) Delete a JSON file
C) Write a dict to a file
D) Convert a DataFrame to JSON

35. Safest way to read an optional 'priority' key from a ticket dict:

A) ticket.get('priority', 'Not set')
B) ticket.priority
C) ticket['priority']
D) ticket.find('priority')

36. How do you count the words in a string variable text?

A) len(text)
B) text.words()
C) text.count()
D) len(text.split())

37. A JSON object { } becomes which Python type after loading?

A) list
B) dict
C) tuple
D) set

38. Most of the world's data is generally considered to be:

A) Unstructured
B) Semi-structured
C) Relational
D) Structured

— Data Collection & Storage —

39. Which built-in Python module works with SQLite databases?

A) sqlalchemy
B) sqlite3
C) mysql
D) pandasql

40. What happens with sqlite3.connect('cinema.db') if the file does not exist?

A) Error is raised
B) Python installs SQLite
C) It connects to the internet
D) A new database file is created

41. Which object is used to execute SQL statements?

A) Connection
B) DataFrame
C) Cursor
D) Series

42. Which call permanently saves inserted rows?

A) cur.fetchall()
B) conn.close()
C) cur.save()
D) conn.commit()

43. cur.fetchall() returns:

A) A dict of columns
B) A single value
C) A DataFrame
D) A list of rows (tuples)

44. Why use CREATE TABLE IF NOT EXISTS?

A) To insert sample rows
B) To speed up queries
C) To delete the old table
D) To avoid an error when the table already exists

45. A PRIMARY KEY column:

A) Can contain duplicates
B) Stores only text
C) Is optional decoration
D) Uniquely identifies each row

46. Which SQL keyword reads data from a table?

A) SELECT
B) UPDATE
C) CREATE
D) INSERT

47. Which loads SQL query results directly into a DataFrame?

A) pd.read_sql_query(query, conn)
B) pd.read_csv(conn)
C) cur.fetchdf()
D) conn.to_dataframe()

48. A central store of cleaned, historical data used for business reporting is a:

A) Cache
B) Data lake
C) Text file
D) Data warehouse

49. A data lake mainly stores:

A) Only cleaned SQL tables
B) Raw data in any format
C) Only images
D) Only one department's data

50. A user uploading an Excel sheet through a web form is an example of collecting data from:

A) Application databases
B) Organizational data warehouse
C) User input files
D) Sensors

Answer Key (1 – 50)

Q	Ans	Note	Q	Ans	Note
1	B	NumPy provides the ndarray.	26	B	
2	B	(10+20+30)/3 = 20.0	27	B	
3	A	NumPy multiplies element-wise (vectorized).	28	C	
4	D	Arrays are homogeneous.	29	D	
5	A	import pandas as pd	30	A	Keys give structure, but schema is flexible and nested.
6	D	A Series is 1-D; a DataFrame is 2-D.	31	C	
7	A	Default n=5.	32	D	
8	C	e.g. (8, 5).	33	B	
9	A	Multiple columns need a list inside brackets.	34	A	json.loads() reads from a string.
10	D	Boolean indexing.	35	A	Avoids KeyError when the key is missing.
11	D	Use &, |, ~ with parentheses.	36	D	len(text) counts characters.
12	B	iloc is position-based.	37	B	A JSON array [ ] becomes a list.
13	B		38	A	Text, images, audio, video dominate.
14	A		39	B	No installation needed.
15	A	Split–apply–combine.	40	D	
16	B	count() gives one total; value_counts() gives per-category counts.	41	C	
17	A		42	D	
18	C	count, mean, std, min, quartiles, max.	43	D	
19	D		44	D	
20	C		45	D	
21	C		46	A	
22	A	Bars compare categories.	47	A	
23	B		48	D	
24	D		49	B	Structured, semi-structured and unstructured.
25	B		50	C	

Score yourself: 45–50 excellent • 35–44 good, revise weak topics • below 35 re-read the notes and re-run your practice scripts.